Contents

Manuale d'uso — Pipeline Dataverse -> Archivematica **2**
Indice . . . 3
1. Panoramica del flusso . . . 3
2. Prerequisiti . . . 4
Software . . . 4
Accesso a Dataverse . . . 4
Accesso ad Archivematica . . . 4
Permessi filesystem . . . 4
Note su SSL . . . 4
3. Configurazione — file .env . . . 5
Contenuto completo del .env . . . 5
Recuperare i valori mancanti . . . 6
Priorita' delle variabili . . . 6
Sicurezza . . . 7
Errori comuni nel .env . . . 7
4. Script 1 — scarica_dataverse.py . . . 7
Cosa fa . . . 7
Controllo dataset vuoti . . . 8
Conservazione dell'originale (file tabellari) . . . 8
Struttura a cartelle e unicità dei nomi . . . 9
Verifica di integrità dei file . . . 10
Esportazione DCAT-AP e Schema.org . . . 11
Opzioni da riga di comando . . . 12
Esempi . . . 12
Output a schermo . . . 12
Comportamento idempotente . . . 14
5. Script 2 — archivematica_ingest.py . . . 15
Cosa fa . . . 15
Gestione di timeout ed errori di rete transitori . . . 15
Sorgente del processing config: copia locale versionata (RACCOMANDATO) . . . 16
Scelte richieste nella processing configuration . . . 16
Opzioni da riga di comando . . . 18
Esempi . . . 20
Output a schermo . . . 20
6. Struttura del pacchetto e versioni del dataset . . . 21
Versioni, major e minor release in Dataverse . . . 21
Il criterio: che cosa si scarica a ogni versione . . . 22
Perché i file si scaricano una volta per major . . . 23
Struttura del pacchetto . . . 24
Il manifest files.json . . . 25
Rinomine e spostamenti di file . . . 27
Ricostruire una versione dal pacchetto . . . 27
Quando la regola non vale . . . 28

I metadati di ogni versione 28
Formato metadata.csv (radice del pacchetto) 29
7. File di stato e report 30
riepilogo_download.json 30
doi_vuoti.json 31
stato_archivematica.json 31
Valori di status (stato_archivematica.json) 32
Riconciliazione dello stato — riconcilia_stato.py 32
Reset manuale di un pacchetto 32
Reset completo 33
8. Risoluzione dei problemi comuni 33
MissingSchema / URL non valido 33
302 Found sulla porta 8000 33
401 Unauthorized 33
404 Not Found sulla Transfer Source UUID 33
SSL Certificate Verify Failed 33
[ANOMALIA] in una minor release 34
EXPORT_DCAT=true nel .env ma dcat.json non viene generato 34
[RISCARICO] ripetuti o “checksum non combacia” su un file . . 34
[FALLBACK] salvata derivata .tab 35
Ho ri-scaricato e mi ritrovo sia i .csv/.dta sia i vecchi .tab . . . 35
[ERRORE] start_transfer HTTP 500: Filepath … does not exist 35
Nel METS tutti i file risultano Unknown 35
I nomi dei file nell’AIP sono diversi da quelli originali 36
[ERRORE] Impossibile leggere il processing config 36
[ERRORE] Copia di processingMCP.xml nel pacchetto fallita (Permission denied sulla DESTINAZIONE) 36
Transfer REJECTED al riavvio 37
Read timed out durante transfer/ingest 37
Transfer “zombie” in Dashboard (in attesa, non rimovibili) . . . 37
Permission denied sulla copia di processingMCP.xml nei pacchetti (cartelle di proprieta’ altrui) 38
400 “Unable to determine the status of the unit” 39
Auto-approve non funziona 39
Elasticsearch non raggiungibile 39
9. Esempio di esecuzione completa 39
Configurazione iniziale (una tantum) 39
Esecuzione standard 39

Manuale d’uso — Pipeline Dataverse -> Archivematica

Questo manuale documenta l’uso di due script Python che automatizzano il trasferimento di dataset da **Dataverse UNIMI** (`dataverse.unimi.it`) ad

Archivematica per la conservazione a lungo termine.

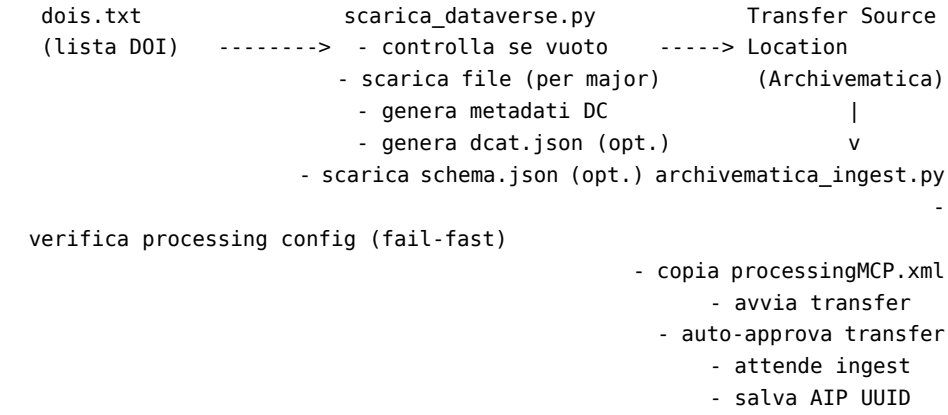
Script	Funzione
`scarica_dataverse.py`	Scarica dataset e metadati da Dataverse e li organizza in pacchetti compatibili con Archivematica
`archivematica_ingest.py`	Trasferisce ed esegue l'ingest dei pacchetti in Archivematica, con approvazione automatica del transfer
`riconcilia_stato.py`	Riconcilia i pacchetti "failed" del file di stato con gli AIP realmente presenti nello Storage Service (es. dopo approvazioni manuali)

Indice

1. Panoramica del flusso
2. Prerequisiti
3. Configurazione — file .env
4. Script 1 — scarica_dataverse.py
5. Script 2 — archivematica_ingest.py
6. Struttura del pacchetto e versioni del dataset
7. File di stato e report
8. Risoluzione dei problemi comuni
9. Esempio di esecuzione completa

---

1. Panoramica del flusso



Il flusso tipico e':

1. Si prepara `dois.txt` con l'elenco dei DOI da archiviare.
2. Si esegue `scarica_dataverse.py`: per ogni DOI recupera le versioni pubblicate, controlla che il dataset contenga file e costruisce il pacchetto nella Transfer Source Location di Archivematica. I metadati vengono conservati per ogni versione, i file una sola volta per ciascuna major release (sezione 6).
3. Si esegue `archivematica_ingest.py`: avvia il transfer e l'ingest per ogni pacchetto non ancora processato, in modo completamente automatico.
4. Lo stato viene salvato in file JSON per le esecuzioni successive.

---

2. Prerequisiti

Software

```
pip install requests --break-system-packages
```

Accesso a Dataverse

- Una **API key Dataverse** (necessaria solo per dataset ad accesso ristretto; per dataset pubblici puo' essere lasciata vuota).

Accesso ad Archivematica

- **Dashboard API key** dell'utente Archivematica (es. `zfed`)
- **Storage Service API key** dell'utente Storage Service (es. `archivematica`)
- **UUID della Transfer Source Location**
- Accesso in lettura/scrittura alla Transfer Source Location sul filesystem

Permessi filesystem

```
sudo usermod -aG archivematica <utente>
# Richiede logout/login per avere effetto
```

Note su SSL

La pipeline comunica con due servizi diversi, e la verifica del certificato si configura separatamente per ciascuno.

Archivematica. Le installazioni locali e di test usano spesso un certificato self-signed, che la verifica standard rifiuterebbe. In questo caso si imposta `SSL_VERIFY=false` nel `.env`, oppure si usa `--no-verify-ssl` con `archivematica_ingest.py`.

Dataverse. `dataverse.unimi.it` espone un certificato valido, firmato da un'autorità pubblica: la verifica deve restare attiva, ed è il comportamento

predefinito. Gli script che interrogano Dataverse (`scarica_dataverse.py`, `analizza_versioni.py`, `verifica_raggiungibilita.py`, `censisci_archivi.py`, `diagnostica_checksum.py`) leggono una variabile propria, `DATAVERSE_SSL_VERIFY`, che vale `true` se non è impostata, e ignorano `SSL_VERIFY`.

La separazione corregge un difetto delle versioni precedenti degli script. Con un'unica variabile, il `SSL_VERIFY=false` impostato per Archivematica nel `.env` condiviso disattivava anche la verifica verso Dataverse: le connessioni a `dataverse.unimi.it` partivano senza alcuna garanzia di raggiungere davvero quel server. Disattivare la verifica verso Dataverse ha senso solo in casi particolari, per esempio dietro un proxy che intercetta il traffico TLS: si imposta allora `DATAVERSE_SSL_VERIFY=false`, oppure si passa `--no-verify-ssl` a `scarica_dataverse.py`.

URL tipici per le API:

- Dashboard: `https://<host>` (porta 443)
- Storage Service: `https://<host>:8000`

---

3. Configurazione — file .env

Tutti i parametri vanno nel file `.env` nella stessa cartella degli script. Il file viene caricato automaticamente all'avvio senza dipendenze esterne.

```
cp .env.template .env
nano .env
```

Contenuto completo del .env

```
# ── Dataverse ──────────────────────────────────────────────
# API key (lasciare vuoto per dataset pubblici)
DATAVERSE_API_KEY=

# Verifica del certificato di dataverse.unimi.it. Il certificato e'
# valido: lasciare true. Distinta da SSL_VERIFY (Archivematica).
DATAVERSE_SSL_VERIFY=true

# Cartella di destinazione (Transfer Source Location di Archivematica)
# Recuperabile con: python3 archivematica_ingest.py --list-sources
OUTPUT_DIR=/mnt/e/DROPBOX/Dropbox/_ARKIVE/TRANSFER_SOURCE

# Esportazione DCAT-AP (dcat.json, per ogni versione) e Schema.org
# (schema.json, solo per l'ultima versione pubblicata).
# Equivalente al flag --dcat da CLI.
EXPORT_DCAT=false

# ── Archivematica Dashboard ────────────────────────────────
```

```
AM_URL=https://192.168.139.39
AM_USER=zfed
AM_API_KEY=

# ── Archivematica Storage Service ──────────────────────────
SS_URL=https://192.168.139.39:8000
SS_USER=archivematica
SS_API_KEY=

# ── Transfer ───────────────────────────────────────────────
AM_TRANSFER_SOURCE_UUID=
AM_TRANSFER_TYPE=standard
AM_PROCESSING_CONFIG=dataverse_001

# Copia locale VERSIONATA del processing config XML (RACCOMANDATO).
# Elimina la dipendenza dalla shared directory di Archivematica
# (/var/archivematica/...), i cui permessi possono cambiare con
# update/riavvii. Setup: vedi sezione 5.
AM_PROCESSING_MCP_PATH=/home/zfed/arkive/config/dataverse_001ProcessingMCP.xml

AM_AUTO_APPROVE=true

# Timeout API in secondi (default: 120). Con dataset grandi o
# Archivematica sotto carico servono valori generosi; gli errori
# di rete transitori vengono comunque ritentati.
AM_API_TIMEOUT=120

# ── SSL ────────────────────────────────────────────────────
# Riguarda solo Archivematica (certificati self-signed).
# Gli script che interrogano Dataverse usano DATAVERSE_SSL_VERIFY.
SSL_VERIFY=false
```

Recuperare i valori mancanti

```
# UUID Transfer Source
python3 archivematica_ingest.py --no-verify-ssl --list-sources

# Processing configuration disponibili
python3 archivematica_ingest.py --no-verify-ssl --list-processing-configs
```

Priorita' delle variabili

Priorita'	Fonte
1 (massima)	Argomento da riga di comando
2	Variabile d'ambiente di sistema (`export`)

Priorita'	Fonte
3	File `.env`
4	Default nel codice

> **Attenzione:** se in una sessione precedente hai eseguito `export $(grep -v '^#' .env | xargs)`, le variabili rimangono attive e hanno priorita' sul `.env`. In caso di comportamenti inattesi, aprire una nuova sessione shell.

Sicurezza

```
echo ".env" >> .gitignore
```

Errori comuni nel .env

Errore	Sintomo	Correzione
`SS_URL=https//...`	`MissingSchema`	`SS_URL=https://...`
`SS_URL=http://...`	`302 Found`	`SS_URL=https://...`
Chiavi API vuote	`401 Unauthorized`	Compilare le chiavi
UUID errato	`404 Not Found`	Usare `--list-sources`
Variabili esportate in sessione precedente	Valori vecchi	Nuova sessione shell

---

4. Script 1 — scarica_dataverse.py

Cosa fa

Per ogni DOI elencato in `dois.txt`:

1. Recupera l'elenco delle versioni del dataset ed esclude l'eventuale bozza (`DRAFT`), che non è una versione pubblicata (sezione 6).
2. **Controlla se il dataset contiene file**: se è vuoto, non crea la directory e registra il DOI in `doi_vuoti.json`.
3. Per ciascuna versione pubblicata salva i metadati: `dataverse.json`, il `metadata.csv` Dublin Core della versione, il manifest dei file `files.json` e, se `EXPORT_DCAT=true`, `dcat.json` e `schema.json` (il secondo solo per l'ultima versione pubblicata).
4. Scarica i file **una sola volta per ciascuna major release**, nella prima versione pubblicata della major (di norma `x.0`). Per i file **ingeriti come tabellari** scarica l'**originale** depositato (`format=original`) e non la derivata `.tab`, e **verifica l'integrità** di ogni file contro il checksum (o la dimensione) dichiarato da Dataverse. Le minor release successive non scaricano file: il loro `files.json` rimanda ai contenuti già conservati. Il criterio, le sue ragioni e le sue garanzie sono descritti nella sezione 6.

5. Genera un `metadata.csv` complessivo nella radice del pacchetto, letto da Archivematica per il METS `dmdSec`.

Controllo dataset vuoti

Prima di creare qualsiasi directory, lo script verifica che il dataset contenga almeno un file. Se tutte le versioni sono prive di file:

- la directory del DOI **non viene creata**
- il DOI viene registrato in `doi_vuoti.json` con data e motivo
- il riepilogo mostra `Vuoti: N` nel conteggio finale

Il controllo considera solo le versioni pubblicate. Con una API key, l'endpoint delle versioni restituisce anche la bozza aperta, ma questa viene esclusa prima del controllo: un dataset che ha file soltanto nella bozza risulta quindi vuoto, coerentemente con il fatto che quei file non sono ancora stati pubblicati.

Cause tipiche di dataset vuoto:

- dataset ancora in bozza (non ancora pubblicato)
- accesso ristretto (embargo attivo)
- dataset pubblicato ma senza file allegati

Conservazione dell'originale (file tabellari)

Quando un file viene caricato su Dataverse in un formato tabellare riconosciuto (Stata `.dta`, SPSS `.sav`, CSV, xlsx…), Dataverse lo **ingerisce**: conserva il file originale depositato e ne genera una rappresentazione derivata `.tab` (tab-delimited), che è quella servita di default dall'API di accesso.

Ai fini della conservazione questa distinzione è decisiva. L'oggetto autentico da preservare secondo OAIS è il **bitstream originale depositato dal ricercatore**, non la normalizzazione prodotta da Dataverse: è Archivematica, in fase di ingest, a generare le derivate di preservazione e accesso secondo il proprio FPR. Inoltre la derivata `.tab` viene **rigenerata a ogni richiesta** e ha un checksum instabile, mentre l'originale è statico e verificabile.

Per questo, quando `_is_ingested_tabular` riconosce un file ingerito (presenza dei campi `originalFileName`/`originalFileFormat`), la pipeline:

1. scarica l'originale con `…/api/access/datafile/{id}?format=original`;
2. lo salva con il **nome originale** (`02500.csv`, `dati.dta`…), non `.tab` — fondamentale perché Archivematica identifichi correttamente il formato (Siegfried/FITS) e pianifichi la conservazione giusta;
3. ne verifica l'integrità con il checksum del manifest.

Fallback: se `format=original` non è disponibile (alcune versioni Dataverse rispondono `404`), la pipeline ripiega sulla derivata `.tab`, in modalità degradata tracciata a log con `[FALLBACK]`, invece di far fallire il DOI. Sulla `.tab` la verifica del checksum non è applicabile: si controlla solo che il file non sia vuoto.

I file **non** ingeriti (PDF, ZIP, immagini, e ogni file non tabellare) non hanno un originale distinto: vengono scaricati e verificati come sempre, senza `format=original`.

Struttura a cartelle e unicità dei nomi

Il nome con cui un file viene salvato su disco deve riprodurre fedelmente la sua identità in Dataverse, ed evitare che due file distinti finiscano sullo stesso percorso. Sono tre le insidie, tutte fonte di **perdita silenziosa** (il file sovrascritto sparisce senza errore):

1. **`directoryLabel`** — Dataverse organizza i file in sottocartelle tramite questo campo (es. `NMR 1h/13C/pdata/1`). La pipeline lo rispetta: ignorarlo appiattirebbe l'albero e i file omonimi in cartelle diverse si sovrascriverebbero. Preservare la struttura non è un dettaglio estetico — per molti formati scientifici (dati NMR Bruker/Varian, output strumentali) la gerarchia di cartelle *è* parte del dato e senza di essa i file non sono interpretabili.
2. **`originalFileName` non univoco** — due file tabellari distinti possono condividere lo stesso `originalFileName` (con `label` diversi). Poiché l'originale si salva con quel nome, il secondo sovrascriverebbe il primo.
3. **filesystem case-insensitive** — su drvfs/NTFS `KLC1.tif` e `klc1.tif` sono lo stesso file; il secondo sovrascrive il primo pur essendo, per Dataverse, file distinti.

Nei casi 2 e 3 la pipeline **disambigua**: quando il percorso calcolato collide con uno già assegnato nella stessa versione (confronto condotto in minuscolo, per coprire anche il caso 3), il file viene collocato in una sottocartella che porta il `label`, che Dataverse garantisce univoco. La disambiguazione avviene prima dello skip-se-esiste ed è deterministica, così il resume non crea doppioni. `metadata.csv` elenca i file annidati (via `rglob`), altrimenti il SIP risulterebbe incompleto.

Il controllo che intercetta questa classe di errori è il confronto fra il numero di file su disco e quello atteso secondo i metadati dell'API: è l'unico che rileva una sovrascrittura, dato che download e verifica di integrità riportano comunque successo. Lo strumento `trova_da_riscaricare.py` lo applica all'intero scope, prendendo il numero atteso dal report di `verifica_raggiungibilita.py`.

Perché il confronto sia corretto, il report deve contare i file con la stessa regola con cui la pipeline li scarica: solo nella prima versione di ogni major, bozze escluse (sezione 6). I report prodotti prima di questa regola sommavano invece i file di tutte le versioni. Confrontato con un pacchetto attuale, un report del genere farebbe risultare incompleto ogni dataset con minor release, e `--elimina` ne cancellerebbe la cartella. Per evitarlo, ogni report registra il criterio con cui è stato calcolato (campo `criterio_volume`):

- `trova_da_riscaricare.py`, se trova il criterio precedente, lo segnala e rifiuta `--elimina`;
- `prepara_lotti.py` avverte che i lotti risulterebbero sovradimensionati;

- rilanciando `verifica_raggiungibilita.py` sulla stessa lista, i DOI calcolati con il criterio precedente vengono ricalcolati automaticamente.

Verifica di integrità dei file

Ogni file scaricato viene confrontato con i metadati di integrità che Dataverse espone nel `dataFile`. Lo scopo è impedire che un download **troncato** (interruzione, timeout, lock transitori su drvfs/Dropbox) lasci sul disco un file parziale che ai run successivi verrebbe considerato "già presente" e non più ri-scaricato. Un file monco entrerebbe nel SIP, Archivematica ne calcolerebbe un checksum PREMIS valido su un contenuto errato, e finirebbe nell'AIP: una corruzione silenziosa, il difetto peggiore in un contesto di conservazione OAIS/PREMIS.

Per i file ingeriti il checksum del manifest si riferisce all'**originale** (Dataverse lo calcola al momento del caricamento, prima dell'ingest — verificato empiricamente su 30/30 file campionati di JMKSAW su dataverse.unimi.it). Poiché la pipeline scarica proprio l'originale, la verifica è **fixity forte**. È anche il motivo per cui in passato i `.tab` producevano falsi mismatch di massa: si confrontavano i byte della derivata con il checksum dell'originale.

Il controllo usa, in ordine di preferenza:

1. **Dimensione attesa** come controllo di skip primario (per gli ingeriti `originalFileSize`, altrimenti `fileSize`). Un download troncato si manifesta sempre come byte mancanti, quindi la sola dimensione lo intercetta con un costo minimo (una `stat()`), senza rileggere l'intero inventario a ogni esecuzione — aspetto rilevante su drvfs.
2. **Checksum** come fallback di skip quando la dimensione non è disponibile, e soprattutto come **verifica post-download** su ogni file appena scaricato. Il checksum Dataverse viene letto in entrambi i formati esistenti:
   - formato nuovo: `checksum: {"type": "MD5" | "SHA-1" | "SHA-256", "value": "..."}`
   - formato vecchio: `md5: "..."`

   con il `type` mappato all'algoritmo `hashlib` corretto (non si assume MD5: al primo run si verifica dal log, cercando se compare `(MD5 ok)` o `(SHA1 ok)` nei `[SKIP]`).
3. **Comportamento storico** (skip se il file esiste e non è vuoto) come ultima spiaggia, solo quando né dimensione né checksum sono disponibili.

Conseguenze pratiche sui file già presenti:

- file **integro** → `[SKIP] Gia' presente (dimensione ok)` (o `(MD5 ok)` / `(SHA1 ok)`)
- file **incompleto o corrotto** → `[RISCARICO] ...` e il file viene ri-scaricato sovrascrivendo quello difettoso

Se un file appena scaricato **non** supera la verifica post-download, viene **rimosso** e il download è segnalato come errore: la versione risulta `partial` nel `riepilogo_download.json` e nessun file difettoso resta sul disco a essere saltato al run

seguente.

Esportazione DCAT-AP e Schema.org

Quando `EXPORT_DCAT=true` (nel `.env`) o `--dcat` (da CLI), per ogni versione vengono generati due file aggiuntivi:

`dcat.json` — DCAT-AP 2.x JSON-LD generato **localmente** dai metadati gia' scaricati, senza chiamate HTTP aggiuntive. Mapping principale:

Campo DCAT-AP	Sorgente Dataverse
`dct:title`	`title`
`dct:creator`	`author` (nome + affiliazione)
`dcat:contactPoint`	`datasetContact` (vCard)
`dct:description`	`dsDescription`
`dcat:theme`	`subject`
`dcat:keyword`	`keyword`
`dct:issued`	`productionDate` / `distributionDate`
`dct:publisher`	`publisher`
`dct:license`	`license` / `termsOfUse`
`dcat:distribution`	file (formato, dimensione, checksum) — per i file ingeriti descrive l'**originale**
`owl:versionInfo`	numero versione (es. `1.0`)

Per i file ingeriti come tabellari la `dcat:distribution` descrive coerentemente l'oggetto conservato, cioè l'**originale**: `dct:title` = nome originale, `dcat:mediaType` = `originalFileFormat`, `dcat:byteSize` = `originalFileSize`, `dcat:accessURL` con `?format=original`, e checksum con l'algoritmo dichiarato nel manifest (non più forzato a MD5). Per i file non ingeriti la distribution descrive il file così com'è.

`schema.json` — Schema.org JSON-LD scaricato dall'API Dataverse tramite l'exporter `schema.org`. È la vista *di Dataverse* sul dataset e quindi riflette la rappresentazione di accesso (`.tab`), non necessariamente l'originale che la pipeline conserva: va inteso come copia fedele di ciò che il repository pubblica, non come descrizione dell'AIP.

È inoltre presente **solo nella cartella dell'ultima versione pubblicata**. Il limite viene da Dataverse: l'API di export (`/api/datasets/export`) restituisce soltanto l'ultima versione pubblicata, e per ogni versione precedente Dataverse 6.10.1 risponde con un errore `400`:

```
Non-draft version requested (1.0) but for published versions only the
latest (:latest-published) is supported.
```

Lo script quindi chiede l'export solo per l'ultima versione e per le altre lo segnala a log con `[INFO] schema.json non prodotto`: l'assenza di `schema.json` in `objects/v1.0/metadata/`, `v2.0/metadata/` ecc. **non è un errore**. Generarlo localmente per le versioni precedenti sarebbe possibile, ma il risultato sarebbe una nostra ricostruzione dell'exporter di Dataverse, non la sua vista autentica, che è proprio ciò che `schema.json` deve documentare.

Lo stesso limite riguarda l'export nativo di Dataverse conservato in `dataverse.json` (sezione 6, "I metadati di ogni versione"). `dcat.json` e `metadata.csv`, generati localmente, ci sono invece per ogni versione.

Opzioni da riga di comando

Opzione	Default	Descrizione
`--dois <file>`	`dois.txt`	File con l'elenco dei DOI
`--output <cartella>`	dal `.env` (OUTPUT_DIR)	Cartella di destinazione
`--apikey <chiave>`	dal `.env`	API key Dataverse
`--no-verify-ssl`	`false`	Disabilita la verifica SSL verso Dataverse (come `DATAVERSE_SSL_VERIFY=false`)
`--dcat`	`false`	Abilita esportazione DCAT-AP e Schema.org

Esempi

```
# Standard (tutto da .env)
python3 scarica_dataverse.py

# Con DCAT-AP e Schema.org
python3 scarica_dataverse.py --dcat

# Override cartella di output
python3 scarica_dataverse.py \
  --output /mnt/e/DROPBOX/Dropbox/_ARKIVE/TRANSFER_SOURCE

# Senza verifica SSL verso Dataverse (solo casi particolari,
# es. proxy che intercetta TLS: vedi "Note su SSL")
python3 scarica_dataverse.py --no-verify-ssl
```

Output a schermo

L'esempio riguarda due DOI: il primo ha una versione `v1.0`, una minor `v1.1` in cui un file è stato rinominato e una bozza aperta; il secondo è vuoto.

```
============================================================
  Dataverse UNIMI — Download automatico (file alle major release)
  Sorgente DOI : dois.txt (2 DOI)
  Destinazione : /mnt/.../TRANSFER_SOURCE
  API key      : ******5678
  SSL Dataverse: True
  Export DCAT  : True
============================================================

[1/2] DOI: doi:10.13130/RD_UNIMI/ESEMPIO
  → Recupero lista versioni...
  [INFO] Bozza (DRAFT) presente: esclusa dal download.
  → 2 versione/i trovata/e: v1.0, v1.1
  ── Versione v1.0 [RELEASED] — download
    [metadata] dataverse.json salvato
    [metadata] dcat.json generato
    [INFO] schema.json non prodotto: l'export Schema.org di Dataverse
           e' disponibile solo per l'ultima versione pubblicata
    [data] 2 file trovati, inizio download...
      Scarico: README.txt ...
OK (2 KB)
      Scarico: 1960_1969_ivt_ar.nc ...
OK (46 KB)
    [metadata] files.json generato (2 file, 0 rimandati a versioni
               precedenti)
    [metadata] metadata.csv (versione) generato (2 righe)
  ── Versione v1.1 [RELEASED] — solo metadati (contenuti in v1.0)
    [metadata] dataverse.json salvato
    [metadata] dcat.json generato
    [metadata] schema.json salvato
    [data] 2 file: nessun download previsto, verifica rispetto a v1.0...
      [rinomina] id 382839: '1960_1969_ivt_ar.nc'
                 -> '1961_1969_ivt_ar.nc'
                 (contenuto in objects/v1.0/objects/1960_1969_ivt_ar.nc)
    [metadata] files.json generato (2 file, 2 rimandati a versioni
               precedenti)
    [metadata] metadata.csv (versione) generato (1 righe)
  [metadata] metadata.csv generato (2 righe, 2 versione/i)
  [validazione] metadata.csv OK — nessun problema rilevato.
  ✓ Stato: ok | Versioni: 2 | File scaricati/presenti: 2 | Rimandati: 2
    | Errori file: 0 | Bozza esclusa

[2/2] DOI: doi:10.13130/RD_UNIMI/VUOTO
  → Recupero lista versioni...
  [INFO] Dataset vuoto: 1 versione/i trovata/e ma nessun file presente
         (dataset vuoto o accesso ristretto)
```

```
  [INFO] Directory non creata.
  ○ Stato: empty | Versioni: 0 | File scaricati/presenti: 0
    | Rimandati: 0 | Errori file: 0
  Motivo: 1 versione/i trovata/e ma nessun file presente
          (dataset vuoto o accesso ristretto)

============================================================
  RIEPILOGO FINALE
  Completati  : 1
  Parziali    : 0
  Errori      : 0
  Vuoti       : 1
  Con anomalie: 0
  Riepilogo   : .../riepilogo_download.json
  DOI vuoti   : .../doi_vuoti.json
============================================================
```

Le righe più lunghe sono state spezzate per la stampa; a terminale ciascuna occupa una sola riga. L'esito di ogni download (`OK`, `ERRORE`) compare invece davvero sulla riga successiva a `Scarico:`, perché il log scrive ogni messaggio su una riga propria.

La riga di stato di ogni DOI riassume l'esito:

- **File scaricati/presenti** conta i file conservati nel pacchetto a fine esecuzione, sia scaricati in questo run sia già presenti e integri;
- **Rimandati** conta i file delle minor release che rimandano a un contenuto conservato nella versione di riferimento della major (sezione 6);
- **Anomalie: N** compare solo se una minor non ha rispettato la regola (sezione 6, "Quando la regola non vale");
- **Bozza esclusa** compare se il dataset ha una bozza aperta.

Le righe `[INFO] schema.json non prodotto` sono attese per ogni versione che non è l'ultima pubblicata. Nel riepilogo finale, **Con anomalie** conta i DOI con almeno un'anomalia: un valore diverso da zero va sempre esaminato.

Comportamento idempotente

Lo script può essere rieseguito senza duplicare il lavoro già fatto:

- File già scaricati: **saltati solo se superano la verifica di integrità** (vedi "Verifica di integrità dei file"). Un file integro produce `[SKIP] Gia' presente (dimensione ok)`; un file incompleto o corrotto viene invece `[RISCARICO]` e ri-scaricato. Questo rende la ri-esecuzione anche un modo per **sanare** una cartella rimasta con download parziali dopo un'interruzione. Nota: i file ingeriti sono riconosciuti per **nome originale** (`02500.csv`), quindi cartelle popolate da run precedenti a questa modifica — che contengono i vecchi `.tab` — non vengono riconosciute

(vedi troubleshooting)
- Minor release: non scaricano file, quindi rieseguirle non comporta download. Fa eccezione il recupero: se un file non era stato ottenuto nella versione di riferimento della major, la minor successiva ne ritenta il download (sezione 6)
- `dataverse.json`, `dcat.json`, `schema.json`, `metadata.csv` di versione: generati solo se mancanti
- `files.json` di versione: **sempre rigenerato**, perché descrive i percorsi effettivamente presenti sul disco, che cambiano quando un download prima fallito va a buon fine
- `metadata.csv` complessivo: **sempre rigenerato**
- `doi_vuoti.json`: **aggiornato incrementalmente** (le voci precedenti vengono preservate, non sovrascritte)

---

5. Script 2 — archivematica_ingest.py

Cosa fa

All'avvio, PRIMA di elaborare qualunque pacchetto, lo script verifica (**fail-fast**) che il file XML della processing configuration sia leggibile. Se non lo e', si interrompe con exit code 1: senza `processingMCP.xml` i pacchetti verrebbero processati con la configurazione di **default** di Archivematica (es. con scansione virus attiva), e il problema emergerebbe solo a ingest avviato — molto piu' costoso da diagnosticare.

Poi, per ogni pacchetto DOI nella Transfer Source Location non ancora completato:

1. Copia `processingMCP.xml` nella radice del pacchetto e **verifica** che la copia sia avvenuta (in caso contrario il pacchetto viene marcato `failed` e il transfer NON parte)
2. Avvia il **Transfer** con nome `AIP-<YYYYMMDD>-<DOI>`
3. Approva automaticamente il transfer tramite API (`/api/transfer/unapproved/`
   - `/api/transfer/approve/`)
4. Attende il completamento del Transfer (polling)
5. Attende il completamento dell'**Ingest** (SIP -> AIP)
6. Salva l'UUID dell'AIP nel file di stato

Ogni cartella DOI viene trasferita come **un unico pacchetto** con tutte le sue versioni.

Gestione di timeout ed errori di rete transitori

Un timeout di lettura NON significa che l'operazione sia fallita lato Archivematica — significa solo che la risposta non e' arrivata in tempo. Per questo:

- Le chiamate di **polling** e le altre GET vengono ritentate automaticamente (3 tentativi, 15s di pausa) in caso di `Read timed out` o errore di connessione.
- Se **`start_transfer` va in timeout**, la richiesta potrebbe comunque essere arrivata: con dataset grandi la copia nella watched directory puo' superare il timeout. Lo script cerca allora il transfer per nome tra quelli in attesa di approvazione e, se lo trova, si **RIAGGANCIA** e prosegue normalmente. Solo se il transfer non esiste davvero il pacchetto viene marcato `failed`. Questo evita transfer zombie in Dashboard e AIP duplicati al retry.
- Il timeout e' configurabile con `AM_API_TIMEOUT` nel `.env` o `--api-timeout` da CLI (default: 120s).

Se nonostante tutto un transfer viene completato manualmente dalla Dashboard dopo che lo script lo ha dichiarato `failed`, usare `riconcilia_stato.py` (vedi sezione 7) per allineare il file di stato PRIMA del lancio successivo, altrimenti il pacchetto verrebbe riprocessato da zero creando un AIP duplicato.

Sorgente del processing config: copia locale versionata (RACCOMANDATO)

Il file XML puo' essere letto da due sorgenti, in ordine di priorita':

1. **Copia locale versionata** — `--processing-mcp-path` o `AM_PROCESSING_MCP_PATH` nel `.env`. Nessuna dipendenza dalla shared directory di Archivematica, i cui permessi possono cambiare con update, riavvii dei servizi o salvataggi della configurazione dal Dashboard. La copia puo' essere committata nel repository ed e' riutilizzabile su altri server della pipeline.
2. **Fallback legacy** — `<PROCESSING_CONFIGS_DIR>/<nome>ProcessingMCP.xml` nella shared directory di Archivematica, usando il nome passato con `--processing-config`.

Setup una tantum della copia locale:

```
mkdir -p ~/arkive/config
sudo cp /var/archivematica/sharedDirectory/sharedMicroServiceTasksConfigs/processingMCPConfigs/datave
sudo chown $(whoami): ~/arkive/config/dataverse_001ProcessingMCP.xml
```

e nel `.env`:

```
AM_PROCESSING_MCP_PATH=/home/<utente>/arkive/config/dataverse_001ProcessingMCP.xml
```

> **Attenzione:** se la processing configuration viene modificata nel Dashboard di Archivematica, la copia locale va riesportata con il comando `sudo cp` qui sopra. E' un evento raro e consapevole, ma va ricordato. Inoltre, dopo aver modificato il `.env`, riesportare l'ambiente nelle sessioni shell attive: `export $(grep -v '^#' .env | xargs)`

Scelte richieste nella processing configuration

Non tutte le scelte sono indifferenti: due incidono direttamente sulla qualita' archivistica degli AIP e vanno verificate **prima** di lavorazioni estese.

Decisione	Valore richiesto	Perche'
Perform file format identification (**Transfer**)	**Yes**	Senza, nessun file viene identificato e il METS riporta `Unknown` per tutti gli oggetti: niente PUID, niente pianificazione della conservazione, niente monitoraggio dell'obsolescenza
Perform file format identification (**Ingest**)	*No, use existing data*	Corretto: riusa i risultati del transfer invece di rifare il lavoro. **Ma vale solo se l'identificazione in fase di transfer e' attiva**: se e' saltata, non esistono dati da riusare
Normalize	**Normalize for preservation**	Genera le derivate di preservazione secondo il FPR. E' Archivematica a doverle produrre, non il repository di origine
Approve normalization	Yes	Altrimenti il processo si ferma in attesa di approvazione manuale, vanificando l'automazione

Nelle versioni recenti di Archivematica la scelta dello **strumento** di identificazione non compare piu' nella processing configuration (l'opzione e' solo Yes/No): lo strumento si configura in *Preservation Planning* (FPR). Quale sia stato effettivamente usato si legge a posteriori nel METS, negli eventi PREMIS di `format identification` (es. `program="Siegfried"; version="1.11.2"`).

Non modificare l'XML a mano. Gli UUID delle catene (`goToChain`) variano tra versioni di Archivematica: un valore errato produce fallimenti oscuri. Il file va generato dal Dashboard (*Administration -> Processing configuration -> Edit -> Save -> Download*) e poi copiato nella posizione della copia locale versionata.

Verifica rapida che le due scelte critiche siano attive:

```
CFG=~/arkive/config/dataverse_001ProcessingMCP.xml
# identificazione in transfer: la goToChain NON deve essere quella di "Skip"
grep -A2 "f09847c2-ee51-429a-9478-a860477f6b8d" $CFG
# normalizzazione (due punti di decisione, devono avere la stessa catena)
```

```
grep -A2 "cb8e5706-e73f-472f-ad9b-d1236af8095f" $CFG
grep -A2 "7509e7dc-1e1b-4dce-8d21-e130515fce73" $CFG
```

Nota: il file scaricato dal Dashboard **non contiene i commenti** descrittivi (`<!-- Normalize: ... -->`) che rendono leggibile la configurazione. La tabella qui sopra vale come documentazione delle scelte attese.

Opzioni da riga di comando

Sorgente e stato

Opzione	Default	Descrizione
`--source <cartella>`	da Transfer Source	Directory con i pacchetti. Deve trovarsi **dentro** la Transfer Source Location; puo' essere una sottocartella (vedi sotto)
`--state-file <file>`	`stato_archivematica.json`	File di tracciamento

> **Dove devono stare i pacchetti.** Il transfer viene avviato passando ad Archivematica `base64(<location_uuid>:<path_assoluto>)`, e lo Storage Service accetta solo percorsi **interni alla Transfer Source Location**: un pacchetto fuori da li' non viene visto (errore `Filepath ... does not exist`).
>
> Le **sottocartelle sono supportate** (es. `<transfer_source>/LOTTO_01/doi_10.13130_...`): basta puntarvi con `--source`. E' il modo raccomandato per lavorare a lotti, perche' `--source` omesso fa usare la radice della location, dove lo script raccoglie **tutti** i pacchetti DOI presenti — comprese lavorazioni precedenti che non si intende ripetere.

Dashboard Archivematica

Opzione	Default	Descrizione
`--am-url <url>`	dal `.env`	URL Dashboard
`--am-user <utente>`	dal `.env`	Utente Dashboard
`--am-apikey <chiave>`	dal `.env`	API key Dashboard

Storage Service

Opzione	Default	Descrizione
`--ss-url <url>`	dal `.env`	URL Storage Service
`--ss-user <utente>`	dal `.env`	Utente Storage Service

Opzione	Default	Descrizione
`--ss-apikey <chiave>`	dal `.env`	API key Storage Service

Transfer

Opzione	Default	Descrizione
`--transfer-source <UUID>`	dal `.env`	UUID Transfer Source Location
`--transfer-type <tipo>`	`standard`	Tipo di transfer
`--processing-config <nome>`	`dataverse_001`	Processing configuration
`--processing-mcp-path <file>`	dal `.env`	Copia locale versionata del processing config XML (RACCOMANDATO); ha priorita' sulla shared directory
`--ignore-missing-processing-config`	—	Prosegue anche senza processing config XML leggibile (SCONSIGLIATO: i transfer usano il default di Archivematica)
`--auto-approve`	dal `.env`	Approva automaticamente il transfer

Utility

Opzione	Descrizione
`--list-sources`	Elenca Transfer Source Locations ed esce
`--list-processing-configs`	Elenca processing configuration ed esce
`--no-verify-ssl`	Disabilita verifica SSL
`--poll-interval <sec>`	Intervallo polling (default: 15s)
`--api-timeout <sec>`	Timeout chiamate API (default: 120s, o `AM_API_TIMEOUT` nel `.env`)
`--dry-run`	Mostra cosa farebbe senza eseguire

Esempi

```
# Standard (tutto da .env)
python3 archivematica_ingest.py

# Lista Transfer Source disponibili
python3 archivematica_ingest.py --no-verify-ssl --list-sources

# Anteprima senza eseguire
python3 archivematica_ingest.py --dry-run

# Processing configuration diversa
python3 archivematica_ingest.py --processing-config automated

# Copia locale del processing config esplicita (senza .env)
python3 archivematica_ingest.py --processing-mcp-path ~/arkive/config/dataverse_001ProcessingMCP.xml
```

Output a schermo

```
================================================================
  Archivematica Ingest -- trasferimento automatico dataset
  Sorgente      : /mnt/.../TRANSFER_SOURCE
  Pacchetti DOI : 3
  Dashboard     : https://192.168.139.39  (utente: zfed)
  Storage Svc   : https://192.168.139.39:8000  (utente: archivematica)
  Transfer Src  : 76ab3067-...
  Transfer type : standard
  Processing cfg: dataverse_001
  MCP XML       : /home/zfed/arkive/config/dataverse_001ProcessingMCP.xml
  Auto-approve  : True
  SSL verify    : False
  File di stato : stato_archivematica.json
================================================================

[1/3] doi_10.13130_RD_UNIMI_KS2PUX
  --> Elaborazione: doi_10.13130_RD_UNIMI_KS2PUX
   [debug] processingMCP.xml copiato e verificato in .../processingMCP.xml
    Avvio transfer 'AIP-20260619-doi_10.13130_RD_UNIMI_KS2PUX' ...
    Transfer UUID: <uuid>
    [approve] Attendo 10s ... (tentativo 1/5)
    [approve] Trovato: AIP-20260619-...-<uuid> -- invio approvazione ...
    [approve] Approvato (UUID: <uuid>)
    Attendo completamento transfer ...
    [transfer] stato: PROCESSING -- attendo 15s ...
    SIP UUID: <uuid>
    Attendo completamento ingest ...
```

```
    OK Ingest completato -- AIP UUID: <uuid>

============================================================
  RIEPILOGO FINALE
  Completati   : 3
  Gia' presenti: 0
  Errori       : 0
============================================================
```

---

6. Struttura del pacchetto e versioni del dataset

Un dataset Dataverse non è un oggetto fisso: nel tempo riceve nuove versioni, e il pacchetto preparato per Archivematica deve conservarle tutte. Questa sezione spiega prima come Dataverse gestisce le versioni, poi come la pipeline le riporta nel pacchetto: che cosa scarica a ogni passaggio di versione, e perché; come si collega ogni versione ai file conservati; come si ricostruisce una versione; che cosa succede quando un dataset non rispetta la regola.

Versioni, major e minor release in Dataverse

Le versioni di un dataset sono numerate `major.minor`. Ogni modifica a un dataset pubblicato apre una **bozza** (`DRAFT`); al momento della pubblicazione la bozza diventa una nuova versione, e chi pubblica sceglie se numerarla come **minor release** (da `1.0` a `1.1`) o come **major release** (da `1.1` a `2.0`).

La scelta non è del tutto libera. Dataverse impone una major release quando nella bozza sono stati **aggiunti o rimossi file**, oppure quando la versione precedente era stata ritirata; negli altri casi lascia scegliere. Per capire che cosa può cambiare in una minor release serve il modello dati di Dataverse, che descrive un file su due livelli:

- il **file vero e proprio** (`dataFile`), con il suo identificativo `dataFile.id`, il contenuto e il checksum. È condiviso da tutte le versioni in cui il file compare e resta lo stesso finché il file non viene sostituito;
- i **metadati del file in una versione**: il nome (`label`), la cartella (`directoryLabel`), la descrizione. Appartengono alla singola versione e possono cambiare anche in una minor release.

Ne segue che fra due versioni della stessa major lo stesso `dataFile.id` indica lo stesso contenuto, anche se nel frattempo il file ha cambiato nome o cartella.

La regola è stata verificata sul corpus con `analizza_versioni.py`, sui 717 DOI della lista del 30/09/2026. Su 523 passaggi da una versione alla minor successiva, nessuno ha aggiunto, rimosso o modificato un file. Gli unici cambiamenti sui file sono stati 11 **rinomine o spostamenti** (cambi di `label` o `directoryLabel`), in 7 dataset. Resta un caso che la documentazione di Dataverse non regola in modo esplicito: la **sostituzione** di un file (*Replace*), che produce un nuovo

`dataFile.id`. Nel corpus non compare in nessuna minor release, ma la pipeline la intercetterebbe comunque (vedi "Quando la regola non vale").

Due stati richiedono un trattamento a parte:

- **Bozze.** Con una API key, l'endpoint delle versioni restituisce anche la bozza aperta, se c'è: nella lista del 30/09/2026 ne avevano una 28 dataset. La bozza non è una versione pubblicata, non ha un numero e può ancora cambiare: la pipeline la esclude e non la conserva. Le versioni precedenti dello script la scaricavano in una cartella `v0.0`.
- **Versioni ritirate.** Le versioni `DEACCESSIONED` sono per ora elaborate come le altre e segnalate a log con `[AVVISO]`. Il loro trattamento è ancora da decidere.

Il criterio: che cosa si scarica a ogni versione

Per ogni versione pubblicata la pipeline decide che cosa scaricare in base al tipo di passaggio rispetto alla versione precedente: passaggio di major o passaggio di minor. Le due modalità differiscono per un solo aspetto, ma decisivo: i metadati si scaricano sempre, i file solo a ogni nuova major.

Passaggio di major: metadati e file. Cambia il numero prima del punto (`1.2` → `2.0`); vale come passaggio di major anche la prima versione pubblicata del dataset (`1.0`). La versione apre una nuova major e ne diventa la *versione di riferimento*. La pipeline scarica:

- i **metadati** della versione, in `objects/vX.Y/metadata/`: `dataverse.json`, `metadata.csv`, `files.json` e, se abilitati, `dcat.json` e `schema.json`;
- **tutti i file** del dataset in quella versione, in `objects/vX.Y/objects/`, con le regole della sezione 4: originale per i file tabellari, verifica di integrità contro il checksum, struttura a cartelle di `directoryLabel`.

Nei log e in `riepilogo_download.json` questa modalità si chiama `download`.

Passaggio di minor: solo metadati. Cambia solo il numero dopo il punto (`2.0` → `2.1` → `2.2`): la versione appartiene a una major già conservata. La pipeline scarica:

- i **metadati** della versione, in `objects/vX.Y/metadata/`, esattamente come per una major;
- **nessun file**: la cartella `objects/vX.Y/objects/` non viene creata. Al suo posto il manifest `files.json` elenca i file della versione e indica, per ciascuno, dove è conservato il contenuto, cioè nella versione di riferimento della major.

Nei log e in `riepilogo_download.json` questa modalità si chiama `solo_metadati`. Fanno eccezione i soli casi di salvaguardia, in cui un file viene scaricato anche in una minor (vedi "Quando la regola non vale").

Un esempio con sei versioni pubblicate e una bozza aperta:

Versione	Passaggio	Modalità	Metadati	File
`v1.0`	prima versione	`download`	scaricati	scaricati
`v1.1`	minor	`solo_metadati`	scaricati	rimando a `v1.0`
`v1.2`	minor	`solo_metadati`	scaricati	rimando a `v1.0`
`v2.0`	major	`download`	scaricati	scaricati
`v2.1`	minor	`solo_metadati`	scaricati	rimando a `v2.0`
`v3.0`	major	`download`	scaricati	scaricati
bozza	—	esclusa	no	no

In termini operativi, lo script ordina le versioni pubblicate per numero (`versionNumber`, poi `versionMinorNumber`), dopo aver escluso le bozze, e le elabora dalla più vecchia alla più recente. La prima versione che incontra per ciascun valore di `versionNumber` è la versione di riferimento di quella major ed è elaborata in modalità `download`; tutte le successive con lo stesso `versionNumber` sono elaborate in modalità `solo_metadati`. La versione di riferimento è di norma la `x.0`; se questa non compare fra le versioni pubblicate, il ruolo passa alla prima minor disponibile della stessa major, che scarica quindi i file.

Il criterio ha due conseguenze da tenere presenti.

- **Decide il numero di versione, non il contenuto.** Dataverse permette di pubblicare come major anche una versione che cambia solo i metadati. In quel caso la pipeline scarica comunque tutti i file, che risulteranno identici a quelli della major precedente. È un costo accettato: la major è il solo passaggio in cui Dataverse ammette modifiche ai file, e scaricare a ogni major garantisce che ciascuna contenga tutti i propri file.
- **Le minor vengono comunque verificate.** Decidere in base al numero non significa fidarsi alla cieca della regola: in ogni minor i file dichiarati da Dataverse vengono confrontati con quelli della versione di riferimento, per `dataFile.id` e checksum, e ogni differenza viene scaricata e segnalata (vedi “Quando la regola non vale”).

Perché i file si scaricano una volta per major

Fino alla versione precedente, la pipeline scaricava i file di ogni versione nella cartella di quella versione. Un file invariato fra `v1.0` e `v1.1` veniva quindi scaricato due volte e finiva due volte nell’AIP: due copie identiche, ognuna caratterizzata da Archivematica, ognuna con il proprio record PREMIS. Nessuna informazione in più, ma più spazio, più tempo di download e di ingest, e un AIP più difficile da leggere, in cui lo stesso contenuto compare come oggetti distinti.

Poiché nelle minor release i file non cambiano, scaricarli a ogni versione non aggiunge nulla. Con il criterio attuale ogni contenuto è conservato una sola volta per major, nella versione di riferimento, e le minor ne conservano soltanto i metadati e il manifest che vi rimanda.

Sui 717 DOI della lista del 30/09/2026 l'effetto stimato con `analizza_versioni.py` è il seguente:

Criterio	File scaricati	Volume
Tutte le versioni (precedente)	194.898	1.376 GB
Una volta per major (attuale)	159.344	965 GB

La deduplicazione si ferma alla major release, per scelta. Un file invariato fra `v1.0` e `v2.0` viene scaricato in entrambe: in questo modo ogni major contiene tutti i propri file e non dipende dalle cartelle delle major precedenti. Una deduplicazione per checksum sull'intero dataset ridurrebbe ancora il volume (`analizza_versioni.py` ne stima anche questo scenario), ma legherebbe ogni major ai contenuti di quelle precedenti. La granularità per major è un compromesso fra risparmio di spazio e semplicità del pacchetto.

Struttura del pacchetto

Esempio con un dataset a due major, ciascuna seguita da una minor:

```
TRANSFER_SOURCE/
  doi_10.13130_RD_UNIMI_XXXXXX/
    objects/
      v1.0/                <- riferimento major 1: file scaricati
        objects/           <- file del dataset
          dati.csv
          README.txt
        metadata/          <- metadati di questa versione
          dataverse.json   <- metadati Dataverse (preservation)
          metadata.csv     <- Dublin Core della versione
          files.json       <- manifest dei file della versione
          dcat.json        <- DCAT-AP (se EXPORT_DCAT=true)
      v1.1/                <- minor: solo metadati, niente objects/
        metadata/
          dataverse.json
          metadata.csv
          files.json       <- rimanda a objects/v1.0/objects/
          dcat.json
      v2.0/                <- riferimento major 2: file scaricati
        objects/
          dati.csv
          README.txt
          nuovo.csv
        metadata/
          ...              <- come in v1.0
      v2.1/                <- minor, ultima versione pubblicata
```

```
      metadata/
        dataverse.json    <- con l'export Dataverse completo
        metadata.csv
        files.json        <- rimanda a objects/v2.0/objects/
        dcat.json
        schema.json       <- solo nell'ultima versione pubblicata
  metadata/
    metadata.csv          <- Dublin Core dei file presenti (METS)
  processingMCP.xml       <- copiato da archivematica_ingest.py
riepilogo_download.json   <- riepilogo scaricamenti
doi_vuoti.json            <- DOI senza file
```

Ogni versione pubblicata ha la propria cartella `objects/vX.Y/` con i propri metadati in `metadata/`. Solo le versioni di riferimento hanno anche la cartella dei file `objects/vX.Y/objects/`. Una minor release la ha soltanto in caso di anomalia, e in quel caso contiene solo i file scaricati per salvaguardia (vedi "Quando la regola non vale").

Tutto ciò che sta sotto `objects/` entra nell'AIP come oggetto conservato, compresi i metadati delle versioni e i manifest `files.json`. È questo che rende l'AIP autosufficiente: per sapere quali file compongono una versione, e con quali nomi, non serve interrogare Dataverse.

Il manifest files.json

Ogni versione, major o minor, ha in `objects/vX.Y/metadata/` un `files.json`: l'elenco dei file della versione così come Dataverse li presenta, con l'indicazione del punto del pacchetto in cui ciascun contenuto è conservato. Nelle versioni di riferimento il manifest rimanda alla versione stessa; nelle minor rimanda alla versione di riferimento della major.

L'esempio è il manifest della `v1.1` del dataset usato nell'output a schermo della sezione 4, in cui uno dei due file è stato rinominato:


```json
{
  "doi": "doi:10.13130/RD_UNIMI/ESEMPIO",
  "versione": "v1.1",
  "modalita": "solo_metadati",
  "versione_riferimento": "v1.0",
  "nota": "...",
  "file": [
    {
      "id": 382838,
      "label": "README.txt",
      "directoryLabel": null,
      "checksum": {
        "tipo": "md5",
        "valore": "7f91ea97aa0d48689b660185635444e2"
```

```
      },
      "nome_su_disco": "README.txt",
      "percorso_contenuto": "objects/v1.0/objects/README.txt",
      "versione_contenuto": "v1.0",
      "rappresentazione": "nativo",
      "rinominato": false
    },
    {
      "id": 382839,
      "label": "1961_1969_ivt_ar.nc",
      "directoryLabel": null,
      "checksum": {
        "tipo": "md5",
        "valore": "a9f614ebb9cbec1deee04f04724df190"
      },
      "nome_su_disco": "1960_1969_ivt_ar.nc",
      "percorso_contenuto": "objects/v1.0/objects/1960_1969_ivt_ar.nc",
      "versione_contenuto": "v1.0",
      "rappresentazione": "nativo",
      "rinominato": true
    }
  ]
}
```


In testa al file, `modalita` dice se la versione ha scaricato i file (`download`) o rimanda ad altre versioni (`solo_metadati`), e `versione_riferimento` indica la versione di riferimento della major. Per ogni file, l'elenco `file` riporta:

`id` Il `dataFile.id` di Dataverse. È la chiave che collega le occorrenze di un file nelle diverse versioni: lo stesso `id` indica lo stesso contenuto.

`label`, `directoryLabel` Nome e cartella del file **in questa versione**, come li mostra Dataverse.

`checksum` Algoritmo e valore dichiarati da Dataverse. Permettono di verificare che il contenuto indicato da `percorso_contenuto` sia davvero quello del file.

`nome_su_disco`, `percorso_contenuto` Nome e percorso, relativo alla radice del pacchetto, con cui il contenuto è salvato. Possono differire da `label` per due motivi: il file è stato rinominato dopo il download, oppure è un file tabellare di cui si conserva l'originale con il suo nome (`dati.dta` anziché `dati.tab`, vedi "Conservazione dell'originale" nella sezione 4). Valgono `null` se il download non è riuscito.

`versione_contenuto` Versione in cui il contenuto è stato scaricato.

`rappresentazione` Quale rappresentazione è conservata: `nativo` per i file non ingeriti, `originale` per i tabellari scaricati con `format=original`, `derivata_tab` quando si è dovuto ripiegare sulla derivata.

`rinominato` `true` se `label` o `directoryLabel` sono cambiati rispetto alla versione in cui il contenuto è stato scaricato.

Il manifest viene rigenerato a ogni esecuzione, perché descrive i percorsi effettivamente presenti sul disco.

Rinomine e spostamenti di file

Una rinomina in una minor release non comporta un nuovo download. Il contenuto resta salvato con il nome che aveva nella versione di riferimento; il manifest della minor riporta il nome nuovo in `label`, quello su disco in `nome_su_disco`, e `rinominato: true`. Lo stesso vale per uno spostamento di cartella (`directoryLabel`): il contenuto resta nella cartella in cui è stato scaricato.

Il log segnala ogni rinomina con una riga `[rinomina]`. Un caso reale viene dal dataset GR9D83, dove nella `v2.1` è stato corretto l'intervallo di anni nel nome di un file NetCDF:

```
[rinomina] id 382839: '1960_1969_ivt_ar.nc' -> '1961_1969_ivt_ar.nc'
           (contenuto in objects/v2.0/objects/1960_1969_ivt_ar.nc)
```

Il checksum è rimasto lo stesso: è cambiato solo il nome, e il contenuto conservato in `v2.0` è quello giusto anche per la `v2.1`.

Fra due major release i file vengono invece scaricati di nuovo, con il nome che hanno nella nuova versione. Anche qui `files.json` aiuta a leggere le differenze. In DP08SC `v3.0`, per esempio, il file che Dataverse mostra come `IVD_NMBU shipment 3_preliminary data_AAquantification.tab` è salvato come `IVD_NMBU shipment 2_preliminary data_AAquantification.xlsx`. È un file tabellare: la pipeline ne conserva l'originale depositato, che porta il nome con cui era stato caricato, mentre il `label` è stato rinominato in seguito.

Ricostruire una versione dal pacchetto

Il pacchetto conserva tutte le versioni pubblicate, ma le minor release non contengono file. Per ottenere il contenuto completo di una versione si segue il suo manifest:

1. si apre `objects/vX.Y/metadata/files.json` della versione desiderata;
2. per ogni voce dell'elenco `file`, il contenuto si trova in `percorso_contenuto`, relativo alla radice del pacchetto;
3. il file va presentato con il nome e la cartella della versione, cioè `directoryLabel` e `label`, non con `nome_su_disco`;
4. se serve una verifica, il checksum del contenuto deve coincidere con `checksum.valore`.

Nell'esempio precedente, la `v1.1` si ricostruisce così:

File della `v1.1`	Contenuto da prendere
`README.txt`	`objects/v1.0/objects/README.txt`
`1961_1969_ivt_ar.nc`	`objects/v1.0/objects/1960_1969_ivt_ar.nc`

Per i file tabellari il contenuto è l'originale depositato (`rappresentazione: originale`): se il `label` termina in `.tab`, il file ricostruito è comunque l'originale, non la derivata che Dataverse serve di default. È la conseguenza voluta della scelta di conservazione descritta nella sezione 4.

Poiché manifest e file stanno sotto `objects/`, la stessa procedura vale sull'AIP: la ricostruzione non richiede di interrogare Dataverse.

Quando la regola non vale

La regola delle minor release è garantita da Dataverse solo in parte (per aggiunte e rimozioni di file) ed è stata verificata empiricamente per il resto. La pipeline quindi non la dà per scontata: in ogni minor confronta i file con quelli della versione di riferimento, per `dataFile.id` e checksum, e tratta ogni scostamento in modo da non perdere dati.

File con `id` assente dalla versione di riferimento. È un file che la major non conteneva. Viene scaricato nella minor, in `objects/vX.Y/objects/`, e il log riporta un `[ANOMALIA]` di tipo `file_assente_dal_riferimento`.

File con lo stesso `id` ma checksum diverso. Il contenuto risulta cambiato. Anche in questo caso il file viene scaricato nella minor, con un `[ANOMALIA]` di tipo `contenuto_cambiato`.

File presente nella versione precedente e assente in questa. Non c'è nulla da scaricare, perché il contenuto resta conservato nella versione da cui proviene. Il caso viene comunque registrato come `[ANOMALIA]` di tipo `file_rimosso`, perché anche questo smentisce la regola.

Download fallito nella versione di riferimento. Non è un'anomalia della regola ma un errore di rete o di integrità. La minor successiva ritenta il download e lo segnala con `[AVVISO]`; se riesce, il contenuto viene conservato nella cartella della minor e il manifest punta lì.

Le anomalie compaiono nel log, nella riga di stato del DOI (`Anomalie: N`), nel riepilogo finale (`Con anomalie: N`) e in `riepilogo_download.json`, con tipo, `dataFile.id` e percorso di ogni file coinvolto. Sul corpus attuale non se ne attendono. Un valore diverso da zero non indica un errore della pipeline, ma un dataset che non rispetta la regola, e va esaminato caso per caso (vedi sezione 8, "[ANOMALIA] in una minor release").

I metadati di ogni versione

Ogni versione, major o minor, ha la propria cartella `metadata/` con questi file:

dataverse.json I metadati della versione nel formato nativo di Dataverse, in due parti. `version_metadata` è la descrizione completa della versione restituita dall'endpoint delle versioni: blocchi di metadati ed elenco dei file con i loro metadati. `dataverse_export` è l'export `dataverse_json` del dataset, che per il limite dell'API di export descritto nella sezione 4 è presente solo nell'ultima versione pubblicata; nelle altre vale `null`, e il campo `dataverse_export_nota` ne spiega il motivo. I metadati della versione non vanno persi: sono interamente in `version_metadata`.

metadata.csv Dublin Core della versione. Nelle versioni di riferimento contiene una riga per file; nelle minor release, che non hanno file propri, una sola riga riferita alla versione nel suo complesso.

files.json Il manifest dei file descritto sopra.

dcat.json DCAT-AP JSON-LD generato localmente, solo se `EXPORT_DCAT=true`.

schema.json Schema.org JSON-LD esportato da Dataverse, solo se `EXPORT_DCAT=true` e solo per l'ultima versione pubblicata.

"Ultima versione pubblicata" si intende al momento del download. Se in seguito il dataset riceve una nuova versione, un'esecuzione successiva crea la cartella della nuova versione, mentre quella precedente mantiene il proprio `schema.json` e il proprio `dataverse_export`. Erano corretti per quella versione quando sono stati esportati, e i file di metadati già presenti non vengono riscritti.

Formato metadata.csv (radice del pacchetto)

Il `metadata.csv` nella cartella `metadata/` alla radice del pacchetto è il file che Archivematica legge durante il transfer e traspone nel `METS.xml` come `<dmdSec MDTYPE="DC">`. Contiene una riga per ogni file **fisicamente presente** nel pacchetto: i file delle versioni di riferimento e gli eventuali file scaricati per anomalia in una minor release.

filename	dc.title	dc.identifier	…
`objects/v1.0/objects/dati.csv`	Titolo	doi:10.13130/…	…
`objects/v2.0/objects/dati.csv`	Titolo	doi:10.13130/…	…
`objects/v2.0/objects/nuovo.csv`	Titolo	doi:10.13130/…	…

Le minor release senza file non compaiono: una riga che indicasse una cartella inesistente non avrebbe un oggetto a cui riferirsi. I loro metadati Dublin Core restano nel `metadata.csv` della versione, in `objects/vX.Y/metadata/`.

---

7. File di stato e report

riepilogo_download.json

Generato da `scarica_dataverse.py` nella cartella di destinazione, con il riepilogo completo di ogni DOI dell'ultima esecuzione. L'esempio è la voce del primo DOI dell'output a schermo della sezione 4; per un DOI vuoto `versions` è un elenco vuoto e `empty_reason` ne riporta il motivo, come in `doi_vuoti.json`:


```
[
  {
    "doi": "doi:10.13130/RD_UNIMI/ESEMPIO",
    "status": "ok",
    "versions": [
      {
        "version": "v1.0",
        "state": "RELEASED",
        "modalita": "download",
        "versione_riferimento": "v1.0",
        "status": "ok",
        "files_ok": 2,
        "files_riferiti": 0,
        "files_err": 0,
        "anomalie": [],
        "error": null
      },
      {
        "version": "v1.1",
        "state": "RELEASED",
        "modalita": "solo_metadati",
        "versione_riferimento": "v1.0",
        "status": "ok",
        "files_ok": 0,
        "files_riferiti": 2,
        "files_err": 0,
        "anomalie": [],
        "error": null
      }
    ],
    "error": null,
    "empty_reason": null,
    "bozze_escluse": 1,
    "anomalie": 0
  }
]
```


A livello di DOI, oltre allo stato complessivo (`status`: `ok`, `partial`, `error`, `empty`),

il file riporta quante bozze sono state escluse (`bozze_escluse`) e il totale delle anomalie nelle minor release (`anomalie`).

Per ogni versione:

`modalita`, `versione_riferimento` Se la versione ha scaricato i file (`download`) o rimanda ai contenuti della versione di riferimento della sua major (`solo_metadati`), e quale sia questa versione di riferimento.

`files_ok` File di questa versione presenti nel pacchetto a fine esecuzione: scaricati nel run oppure già presenti e integri. In una minor è diverso da zero solo in caso di anomalia o di recupero di un download fallito.

`files_riferiti` File che rimandano a un contenuto conservato in un'altra versione.

`files_err` File non ottenuti, per download fallito o verifica di integrità non superata. Se è maggiore di zero la versione risulta `partial`.

`anomalie` Elenco degli scostamenti dalla regola delle minor release, ciascuno con `tipo`, `id` e `percorso` (sezione 6, "Quando la regola non vale").

doi__vuoti.json

Generato da `scarica_dataverse.py`. Contiene solo i DOI senza file, aggiornato incrementalmente ad ogni esecuzione:

```
[
  {
    "doi": "doi:10.13130/RD_UNIMI/VUOTO",
    "empty_reason": "1 versione/i trovata/e ma nessun file presente (dataset vuoto o accesso ristrett
    "rilevato_il": "2026-06-19T10:30:00+00:00"
  }
]
```

stato__archivematica.json

Generato da `archivematica_ingest.py`. Traccia lo stato di ogni transfer:

```
{
  "doi_10.13130_RD_UNIMI_KS2PUX": {
    "transfer_name": "AIP-20260619-doi_10.13130_RD_UNIMI_KS2PUX",
    "transfer_uuid": "bde477ff-...",
    "sip_uuid": "a1b2c3d4-...",
    "aip_uuid": "e5f6a7b8-...",
    "status": "ingested",
    "completed_at": "2026-06-19T10:42:13+00:00",
    "error": null
  }
}
```

Valori di status (stato_archivematica.json)

Valore	Significato
`in_progress`	Avviato ma non completato
`transferred`	Transfer ok, ingest non completato
`ingested`	Completato – saltato nelle esecuzioni successive
`failed`	Errore – ritentato da zero al prossimo avvio

Riconciliazione dello stato — riconcilia_stato.py

Quando un pacchetto viene completato con un **intervento manuale** in Dashboard (es. approvazione a mano dopo un timeout dello script), Archivematica lo porta correttamente a AIP ma il file di stato lo registra ancora come `failed`, senza AIP UUID. Un lancio successivo di `archivematica_ingest.py` lo riprocesserebbe da zero, creando un **AIP duplicato** per lo stesso DOI.

`riconcilia_stato.py` risolve: per ogni pacchetto `failed` interroga lo Storage Service, cerca un AIP con status UPLOADED il cui `current_path` contenga il DOI, e — solo se il match e' **univoco** — aggiorna la voce a `ingested` con l'AIP UUID reale. Con 0 o 2+ match la voce non viene toccata e viene segnalata per verifica manuale.

```
# Anteprima senza modifiche (consigliata come primo passo)
python3 riconcilia_stato.py --dry-run

# Applica (crea automaticamente un backup del file di stato)
python3 riconcilia_stato.py

# Solo DOI specifici (frammenti, separati da virgola)
python3 riconcilia_stato.py --dois H4W0JR,NZRX1C
```

Configurazione: legge `SS_URL`, `SS_USER`, `SS_API_KEY`, `SSL_VERIFY` dal `.env` (stessa priorita' della pipeline: CLI > export > `.env`). Prima di ogni scrittura crea `stato_archivematica.json.bak_<timestamp>`. Exit code: 0 tutto riconciliato, 2 se restano voci ambigue.

Reset manuale di un pacchetto

```
python3 -c "
import json
s = json.load(open('stato_archivematica.json'))
s.pop('doi_10.13130_RD_UNIMI_KS2PUX', None)
json.dump(s, open('stato_archivematica.json', 'w'), indent=2)
"
```

Reset completo

```
echo '{}' > stato_archivematica.json
```

---

8. Risoluzione dei problemi comuni

MissingSchema / URL non valido

```
MissingSchema: Invalid URL 'https//...'
```

Aggiungere i due punti: `https://` non `https//`.

302 Found sulla porta 8000

```
SS_URL=https://192.168.139.39:8000
```

401 Unauthorized

Verificare `AM_API_KEY` e `SS_API_KEY` nel `.env`.

404 Not Found sulla Transfer Source UUID

```
python3 archivematica_ingest.py --no-verify-ssl --list-sources
```

Aggiornare `AM_TRANSFER_SOURCE_UUID` nel `.env` con l'UUID corretto per il server in uso (ogni installazione Archivematica ha UUID diversi).

SSL Certificate Verify Failed

Il rimedio dipende dal servizio che ha rifiutato il certificato (vedi "Note su SSL" nella sezione 2).

Se l'errore riguarda **Archivematica**, di norma il certificato è self-signed:

```
SSL_VERIFY=false   # nel .env
# oppure
python3 archivematica_ingest.py --no-verify-ssl
```

Se l'errore riguarda **Dataverse** (`dataverse.unimi.it`), `SSL_VERIFY` non ha effetto. Il certificato di Dataverse è valido, quindi l'errore indica di solito qualcosa che si frappone nella connessione, per esempio un proxy che intercetta il traffico TLS. Conviene prima capire la causa; solo se la si accetta, si disattiva la verifica:

```
DATAVERSE_SSL_VERIFY=false   # nel .env
# oppure
python3 scarica_dataverse.py --no-verify-ssl
```

[ANOMALIA] in una minor release

Una minor release contiene un file assente dalla versione di riferimento della major, un file con contenuto cambiato, oppure ha perso un file presente nella versione precedente. La pipeline ha già reagito nel modo sicuro: i file nuovi o cambiati sono stati scaricati nella cartella della minor, e i file rimossi restano conservati nella versione da cui provengono. Non c'è quindi perdita di dati.

L'anomalia va comunque esaminata, perché indica un dataset che non rispetta la regola su cui si basa il pacchetto. Il dettaglio è nel campo `anomalie` della versione in `riepilogo_download.json`: per ogni caso il `tipo`, il `dataFile.id` e il percorso. Confrontando su Dataverse le due versioni coinvolte si capisce di che modifica si tratta; il significato dei tipi è spiegato nella sezione 6, "Quando la regola non vale".

EXPORT_DCAT=true nel .env ma dcat.json non viene generato

Verificare che non ci siano variabili esportate in precedenza con `export` che sovrascrivono il `.env`:

```
unset EXPORT_DCAT
python3 scarica_dataverse.py --dcat   # oppure rilancia in una nuova sessione
```

[RISCARICO] ripetuti o "checksum non combacia" su un file

Durante il download `scarica_dataverse.py` confronta ogni file con il checksum (o la dimensione) dichiarato da Dataverse. Alcuni casi:

- `[RISCARICO] Dimensione non combacia` su un file **già presente**: la copia locale è più piccola dell'attesa, tipicamente un download interrotto in precedenza. È il comportamento voluto: il file viene ri-scaricato. Nessuna azione necessaria.
- `ERRORE (... non combacia)` su un file **appena scaricato**, che si ripete a ogni run: il download arriva sistematicamente corrotto o troncato. Verificare la stabilità della rete/proxy e, se il file è grande, che non si stia colpendo un limite lato server. Il file difettoso viene rimosso a ogni tentativo, quindi non entra mai nel pacchetto.
- Nel log compare `[SKIP] Gia' presente (nessuna verifica disponibile)`: Dataverse non ha esposto né dimensione né checksum per quel file. Lo skip ricade sul comportamento storico (esiste e non è vuoto); in questo caso l'integrità non è garantita dallo script e va eventualmente verificata a valle.

Per sapere quale algoritmo di checksum usa davvero l'istanza, osservare la dicitura tra parentesi nei messaggi di `[SKIP]` (`(MD5 ok)`, `(SHA1 ok)`, …).

[FALLBACK] salvata derivata .tab

Compare quando, per un file ingerito come tabellare, `format=original` non è disponibile (tipicamente un `404` su alcune versioni Dataverse). La pipeline ripiega sulla derivata `.tab` invece di far fallire il DOI. È una modalità **degradata**: il pacchetto conterrà la `.tab` (non verificabile via checksum) invece dell'originale. Se il messaggio è isolato, è tollerabile; se è sistematico su un'istanza, conviene verificare con l'amministratore Dataverse perché `format=original` non risponde, dato che l'obiettivo di conservazione è l'originale.

Ho ri-scaricato e mi ritrovo sia i `.csv`/`.dta` sia i vecchi `.tab`

Da quando la pipeline conserva gli originali, i file ingeriti vengono salvati col nome originale (`02500.csv`), non più `02500.tab`. Su una cartella di download popolata da un run **precedente** a questa modifica, un nuovo run non riconosce i `.tab` come "già presenti" (cerca `02500.csv`) e scarica gli originali **accanto** ai vecchi `.tab`. Non è un errore del codice, è la conseguenza del cambio di nome. Prima di rilanciare su cartelle già popolate in passato, svuotarle (o rimuovere i `.tab` residui). La migrazione dei pacchetti/AIP già prodotti con i `.tab` è un tema a sé, da gestire con una procedura dedicata.

[ERRORE] start_transfer HTTP 500: Filepath ... does not exist

Archivematica non trova il pacchetto. Cause tipiche:

- il pacchetto **non si trova dentro la Transfer Source Location**: lo Storage Service risolve solo percorsi interni alla location. Spostarlo, oppure indicare la location corretta con `--transfer-source`;
- permessi insufficienti: l'utente di Archivematica deve poter attraversare tutte le directory del percorso (bit `x` sul gruppo) e leggere il pacchetto.

Il percorso effettivamente inviato compare nel log come `[debug] Full path`: confrontarlo con quello reale del pacchetto è il modo più rapido per capire dove si perde. Le sottocartelle della location sono supportate.

Nel METS tutti i file risultano `Unknown`

L'identificazione dei formati non è stata eseguita: verificare le scelte della processing configuration (sezione "Scelte richieste nella processing configuration"). Il caso tipico è *identificazione in transfer* impostata su **Skip** con *identificazione in ingest* su *"No, use existing data"*: la seconda non ha dati da riusare, quindi nessun file viene identificato.

Conseguenze: nessun PUID negli oggetti PREMIS, nessuna pianificazione della conservazione e nessuna normalizzazione (il FPR seleziona le regole in base al formato riconosciuto). Gli AIP prodotti in queste condizioni vanno rifatti.

I nomi dei file nell'AIP sono diversi da quelli originali

È il comportamento atteso di Archivematica: i nomi contenenti spazi o caratteri problematici vengono **sanificati** sul filesystem (`IVD_NMBU shipment 1.xlsx` → `IVD_NMBU_shipment_1.xlsx`) e ogni modifica è registrata con un evento PREMIS `filename change`. Il nome originale resta documentato nel METS nell'elemento `originalName`, quindi l'autenticità è preservata: dall'AIP si risale sempre al nome con cui il file è stato depositato.

[ERRORE] Impossibile leggere il processing config

Dalla versione corrente lo script si interrompe (**fail-fast**) se il file XML della processing configuration non e' leggibile, invece di proseguire e produrre pacchetti processati con la configurazione di default di Archivematica (es. con scansione virus attiva).

Soluzione raccomandata — copia locale versionata, indipendente dai permessi della shared directory di Archivematica:

```
mkdir -p ~/arkive/config
sudo cp /var/archivematica/sharedDirectory/sharedMicroServiceTasksConfigs/processingMCPConfigs/datave
sudo chown $(whoami): ~/arkive/config/dataverse_001ProcessingMCP.xml
# nel .env:
#   AM_PROCESSING_MCP_PATH=/home/<utente>/arkive/config/dataverse_001ProcessingMCP.xml
```

Alternative (fragili: i permessi possono ri-cambiare a ogni update/riavvio di Archivematica):

```
sudo usermod -aG archivematica <utente>
# Poi fare logout e login: la membership nel gruppo ha effetto solo
# sulle NUOVE sessioni. Sessioni tmux/screen di lunga data o job cron
# avviati prima dell'aggiunta NON vedono il gruppo.
```

Per proseguire consapevolmente senza processing config (SCONSIGLIATO): `--ignore-missing-processing-config`.

[ERRORE] Copia di processingMCP.xml nel pacchetto fallita (Permission denied sulla DESTINAZIONE)

Diverso dal caso precedente: qui la sorgente e' leggibile ma la scrittura nella radice del pacchetto fallisce. Il pacchetto viene marcato `failed` e il transfer NON parte; gli altri pacchetti proseguono normalmente. Tipico con Transfer Source su mount Windows/Dropbox (WSL drvfs). Cause:

1. **`processingMCP.xml` gia' presente e non scrivibile** nella cartella (es. residuo di una copia manuale con `sudo`, o attributo read-only di Windows). Lo script rimuove automaticamente la destinazione preesistente prima della copia.
2. **Lock transitorio del client Dropbox** durante la sincronizzazione del file. Lo script ritenta automaticamente 3 volte con 2s di pausa.

Se l'errore persiste dopo i retry automatici:

```
# Verifica il file incriminato
ls -l <TRANSFER_SOURCE>/<doi>/processingMCP.xml

# Elenca i pacchetti falliti nel file di stato
python3 -c "import json; s=json.load(open('stato_archivematica.json')); \
print([k for k,v in s.items() if v.get('status')=='failed'])"
```

Poi rilancia `archivematica_ingest.py`: i pacchetti `failed` vengono ritentati automaticamente da zero.

> **Buona pratica:** durante esecuzioni massive, mettere in PAUSA la sincronizzazione Dropbox (o escludere `TRANSFER_SOURCE` dal sync) riduce drasticamente i lock transitori e le collisioni.
>
> **Nota:** un "Permission denied" puo' presentarsi anche con permessi apparentemente corretti su file e directory: verificare il permesso di attraversamento (`x`) su TUTTE le directory del percorso, eventuali ACL (`getfacl`), filesystem di rete (NFS/CIFS con squash degli UID), e che la sessione da cui gira lo script abbia effettivamente la membership nei gruppi attesa (`id` nella STESSA sessione).

Transfer REJECTED al riavvio

Lo script rileva automaticamente status `failed` e riparte da zero.

Read timed out durante transfer/ingest

Con dataset molto grandi o con Archivematica sotto carico (molti ingest in coda), le risposte API possono superare il timeout. Lo script gestisce automaticamente il caso (retry sulle GET, riaggancio del transfer dopo timeout su start_transfer — vedi sezione 5). Se i timeout sono ricorrenti, aumentare `AM_API_TIMEOUT` nel `.env` (o `--api-timeout`).

Se un pacchetto viene comunque marcato `failed` ma in Dashboard il transfer prosegue e arriva in fondo (anche con approvazione manuale), usare `riconcilia_stato.py` per allineare il file di stato PRIMA del lancio successivo (vedi sezione 7).

Transfer "zombie" in Dashboard (in attesa, non rimovibili)

Transfer rimasti in attesa di approvazione che la Dashboard non riesce a rifiutare o rimuovere (tipicamente dopo timeout o riavvii di MCPServer). Sintomi: le azioni nel menu non hanno effetto, oppure il bollino con il conteggio delle decisioni pendenti resta acceso anche dopo aver nascosto le voci.

Lezione chiave: il flag `hidden` nel database nasconde la voce dalla UI ma NON ferma la lavorazione. Finche' il pacchetto resta fisicamente nelle watched

directories, **MCPServer ricrea i job di approvazione a ogni riavvio** (riconoscibili: `createdTime` coincidente con l'orario del riavvio). La rimozione va fatta nell'ordine giusto:

```
# 1. Identifica UUID e percorso fisico dei transfer da rimuovere
mysql -u archivematica -p MCP -e "
SELECT transferUUID, hidden, currentLocation FROM Transfers
WHERE currentLocation REGEXP '<frammento1>|<frammento2>';"

# 2. FERMA MCPServer (altrimenti ricrea i job mentre pulisci)
sudo systemctl stop archivematica-mcp-server

# 3. Rimuovi le cartelle indicate da currentLocation
#    (%sharedPath% = /var/archivematica/sharedDirectory/)
#    ATTENZIONE: solo quelle, verificando che i nomi contengano i DOI attesi.
#    Gli originali in TRANSFER_SOURCE non vengono toccati (start_transfer
#    lavora su una COPIA).

# 4. Backup e pulizia del database
mysqldump --no-tablespaces -u archivematica -p MCP Transfers Jobs \
    > /tmp/backup_transfers_jobs_$(date +%Y%m%d_%H%M).sql
mysql -u archivematica -p MCP -e "
UPDATE Transfers SET hidden=1 WHERE transferUUID IN ('<uuid1>', '<uuid2>');
UPDATE Jobs SET currentStep=4 WHERE currentStep=1
    AND SIPUUID IN ('<uuid1>', '<uuid2>');"
# (currentStep: 1 = awaiting decision, 4 = failed)

# 5. Riavvia e verifica
sudo systemctl start archivematica-mcp-server
sudo systemctl restart archivematica-dashboard
mysql -u archivematica -p MCP -e \
    "SELECT COUNT(*) FROM Jobs WHERE currentStep = 1;"   # atteso: 0, e deve RESTARE 0
```

Nota su mysqldump: senza `--no-tablespaces` l'utente `archivematica` riceve "Access denied; you need PROCESS privilege" — il flag e' innocuo per il backup di singole tabelle.

Permission denied sulla copia di processingMCP.xml nei pacchetti (cartelle di proprieta' altrui)

Se alcune cartelle nella Transfer Source appartengono a un altro utente (es. `archivematica:archivematica` con permessi 755), la copia di `processingMCP.xml` fallisce con Permission denied nonostante retry. Causa tipica: script lanciati in passato con `sudo -u archivematica` o come root. **Mai lanciare gli script della pipeline con un utente diverso da quello operativo.** Rimedio:

```
# Riporta all'utente operativo la proprieta' delle cartelle fallite
# (basta la radice del pacchetto; il gruppo archivematica resta per la lettura)
sudo chown <utente>:archivematica <TRANSFER_SOURCE>/<doi>/
```

400 "Unable to determine the status of the unit"

Errore transitorio: lo script ritenta automaticamente 2 volte (20s di attesa).

Auto-approve non funziona

1. Verificare `AM_AUTO_APPROVE=true` nel `.env`
2. Verificare che il transfer sia in USER_INPUT nel Dashboard
3. Aumentare `APPROVE_TRANSFER_WAIT` nel codice (default: 10s) se Archivematica e' lenta ad avviare il transfer

Elasticsearch non raggiungibile

```
sudo systemctl start elasticsearch
curl -s http://localhost:9200
```

---

9. Esempio di esecuzione completa

Configurazione iniziale (una tantum)

```
git clone https://github.com/zfed/Arkive.git
cd Arkive/tools/dataverse-archivematica

pip install requests --break-system-packages

cp .env.template .env
nano .env
# Compilare almeno:
#   OUTPUT_DIR, DATAVERSE_API_KEY, AM_API_KEY, SS_API_KEY,
#   AM_TRANSFER_SOURCE_UUID, SSL_VERIFY (Archivematica).
#   DATAVERSE_SSL_VERIFY resta true (default).

# Recupera UUID Transfer Source
python3 archivematica_ingest.py --no-verify-ssl --list-sources

# Verifica processing configuration
python3 archivematica_ingest.py --no-verify-ssl --list-processing-configs
```

Esecuzione standard

```
# 1. Prepara la lista DOI
cat > dois.txt << 'EOF'
```

```
doi:10.13130/RD_UNIMI/KS2PUX
doi:10.13130/RD_UNIMI/NFURUT
doi:10.13130/RD_UNIMI/QBRJNN
EOF

# 2. Scarica i dataset (OUTPUT_DIR letto dal .env)
python3 scarica_dataverse.py

# 2b. Con metadati DCAT-AP e Schema.org
python3 scarica_dataverse.py --dcat

# 3. Verifica i dataset vuoti (se presenti)
cat doi_vuoti.json | python3 -m json.tool

# 4. Ingesta in Archivematica (completamente automatico)
python3 archivematica_ingest.py

# 5. Verifica lo stato
cat stato_archivematica.json | python3 -m json.tool

# 6. (solo se e' servito un intervento manuale in Dashboard)
#    Riconcilia il file di stato con gli AIP reali
python3 riconcilia_stato.py --dry-run && python3 riconcilia_stato.py
```

Per le esecuzioni successive, ripetere i passi 2 e 4: gli script elaborano solo cio' che e' nuovo o non ancora completato.